

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



BÁO CÁO ĐỒ ÁN
CS105.Q22 – ĐỒ HỌA MÁY TÍNH
HK2 – Năm học 2025-2026
TIỂU LUẬN FRACTAL

Giảng Viên: Tiến Sĩ Cáp Phạm Đình Thăng

Nhóm sinh viên thực hiện:

Họ và tên	MSSV
Dương Thị Tú Yến	23521846
Nguyễn Mai Hào	23520446
Nguyễn Thế Luân	23520899

TP. Hồ Chí Minh, tháng 4 năm 2026

Mục lục

1	Tìm hiểu về cách vẽ bông tuyết Vankoch (Koch Snowflake)	1
1.1	Giới thiệu	1
1.2	Ý tưởng tổng quát	1
1.3	Đặc điểm toán học	1
1.4	Source code	2
1.5	Phân tích đoạn mã	3
1.6	Kết quả	4
2	Tìm hiểu về cách vẽ đảo Minkowski	4
2.1	Giới thiệu	4
2.2	Ý tưởng tổng quát	5
2.3	Đặc điểm toán học	5
2.4	Source code	5
2.5	Phân tích đoạn mã	8
2.6	Kết quả	9
3	Tìm hiểu về cách vẽ tam giác Sierpinski (Triangles) và Hình vuông Sierpinski (Carpet)	10
3.1	Tam giác Sierpinski (Sierpinski Triangle)	10
3.1.1	Giới thiệu	10
3.1.2	Ý tưởng tổng quát	10
3.1.3	Đặc điểm toán học	10
3.1.4	Source code	10
3.1.5	Phân tích đoạn mã	11
3.2	Hình vuông Sierpinski (Sierpinski Carpet)	11
3.2.1	Giới thiệu	11
3.2.2	Ý tưởng tổng quát	11
3.2.3	Đặc điểm toán học	12
3.2.4	Source code	12
3.2.5	Phân tích đoạn mã	13
3.3	So sánh Tam giác Sierpinski và Hình vuông Sierpinski	13
3.4	Kết quả	13
3.4.1	Sierpinski Triangle	13
3.4.2	Sierpinski Carpet	13
4	Tìm hiểu và trình bày tập Mandelbrot và Julia Set	15
4.1	Tập Mandelbrot	15
4.1.1	Giới thiệu	15
4.1.2	Ý tưởng tổng quát	15
4.1.3	Đặc điểm toán học	15
4.1.4	Source code	15
4.1.5	Phân tích đoạn mã	17
4.2	Tập Julia	17
4.2.1	Giới thiệu	17
4.2.2	Ý tưởng tổng quát	18
4.2.3	Đặc điểm toán học	18

4.2.4	Source code	18
4.2.5	Phân tích đoạn mã	19
4.3	Mối liên hệ giữa tập Mandelbrot và tập Julia	20
4.4	Kết quả	20
4.4.1	Mandelbrot	20
4.4.2	Julia	21
5	Source code và tài liệu tham khảo	21

1 Tìm hiểu về cách vẽ bông tuyết Vankoch (Koch Snowflake)

1.1 Giới thiệu

Koch Snowflake là một fractal hình học nổi tiếng, được xây dựng từ một tam giác đều ban đầu. Ở mỗi bước lặp, mỗi đoạn thẳng của hình sẽ được chia thành ba phần bằng nhau, sau đó thay thế đoạn ở giữa bằng hai đoạn thẳng tạo thành một đỉnh nhô ra ngoài. Quá trình này được lặp lại trên tất cả các đoạn, tạo nên một đường biên ngày càng phức tạp nhưng vẫn có tính tự đồng dạng.

1.2 Ý tưởng tổng quát

- Bắt đầu với một tam giác đều.
- Xét từng cạnh của tam giác.
- Chia mỗi cạnh thành ba đoạn bằng nhau.
- Thay đoạn ở giữa bằng hai đoạn mới tạo thành một tam giác đều nhỏ hướng ra ngoài.
- Lặp lại quy trình trên cho tất cả các đoạn theo số cấp độ mong muốn.

1.3 Đặc điểm toán học

- Sau mỗi lần lặp, mỗi đoạn thẳng được thay thế bởi 4 đoạn nhỏ hơn, do đó số đoạn tăng theo quy luật:

$$N_n = 3 \cdot 4^n$$

với n là số cấp độ lặp.

- Độ dài mỗi đoạn sau một lần lặp giảm còn $\frac{1}{3}$ so với đoạn trước đó.
- Chu vi của hình tăng theo công thức:

$$P_n = P_0 \left(\frac{4}{3}\right)^n$$

nên khi $n \rightarrow \infty$, chu vi tiến tới vô hạn.

- Diện tích của bông tuyết Koch tăng dần qua các lần lặp nhưng hội tụ đến một giá trị hữu hạn.
- Số chiều fractal của đường cong Koch được xác định bởi:

$$D = \frac{\log 4}{\log 3}$$

1.4 Source code

```
function drawKochSnowflake(level) {
  const size = 350;
  const height = size * Math.sqrt(3) / 2;

  const centerX = canvas.width / 2;
  const centerY = canvas.height / 2;

  const A = { x: centerX, y: centerY - height / 2 };
  const B = { x: centerX - size / 2, y: centerY + height / 2 };
  const C = { x: centerX + size / 2, y: centerY + height / 2 };

  ctx.beginPath();
  ctx.moveTo(A.x, A.y);

  drawKoch(A, B, level);
  drawKoch(B, C, level);
  drawKoch(C, A, level);

  ctx.stroke();
}

function drawKoch(p1, p2, level) {
  if (level === 0) {
    ctx.lineTo(p2.x, p2.y);
    return;
  }

  const dx = (p2.x - p1.x) / 3;
  const dy = (p2.y - p1.y) / 3;

  const pa = { x: p1.x + dx, y: p1.y + dy };
  const pb = { x: p1.x + 2 * dx, y: p1.y + 2 * dy };

  const baseAngle = Math.atan2(dy, dx);
  const angle = baseAngle + Math.PI / 3;
  const length = Math.hypot(dx, dy);

  const peak = {
    x: pa.x + Math.cos(angle) * length,
    y: pa.y + Math.sin(angle) * length
  };

  drawKoch(p1, pa, level - 1);
  drawKoch(pa, peak, level - 1);
  drawKoch(peak, pb, level - 1);
  drawKoch(pb, p2, level - 1);
}
```

```
function render() {
    clearCanvas();

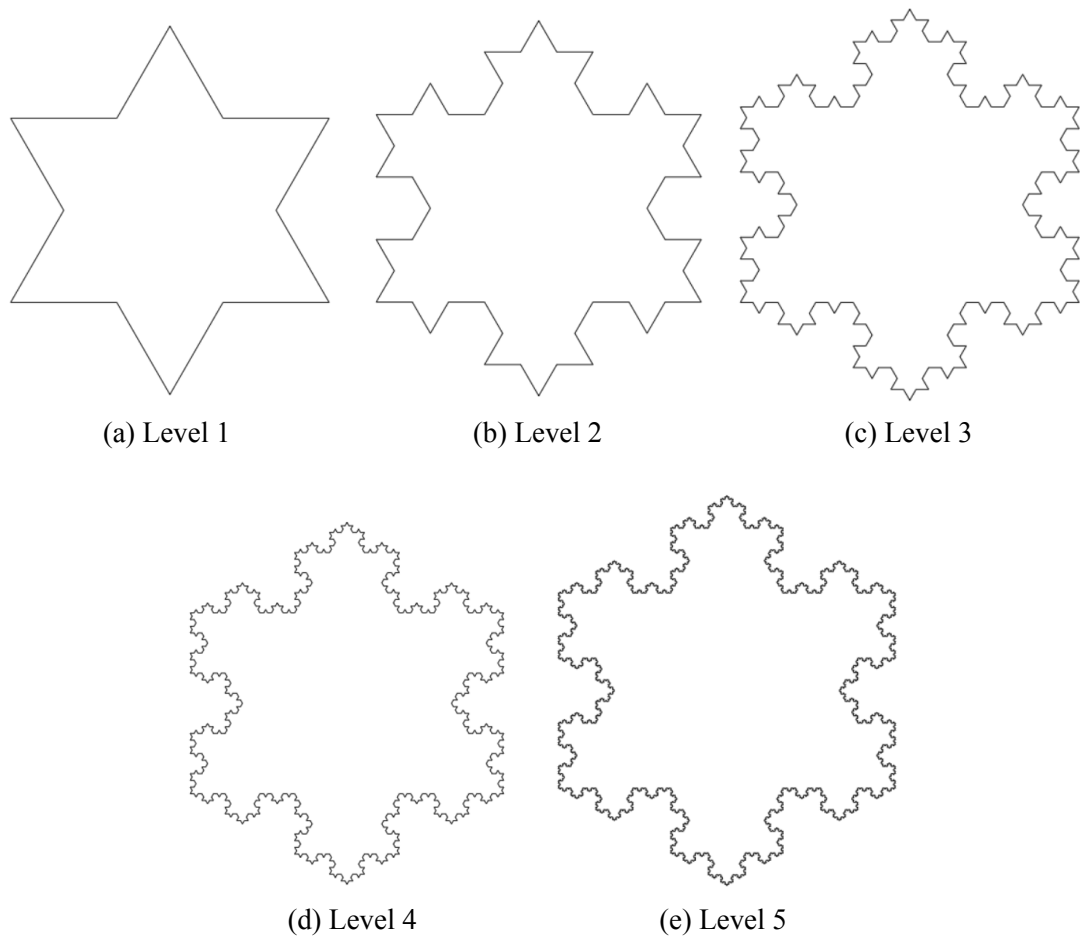
    const level = parseInt(document.getElementById("level").value, 10);
    ctx.strokeStyle = "black";

    drawKochSnowflake(Math.max(0, Math.min(level, 6)));
}
```

1.5 Phân tích đoạn mã

Đoạn chương trình gồm ba thành phần chính: hàm vẽ bông tuyết Koch, hàm đệ quy xử lý từng cạnh và hàm điều khiển hiển thị.

- **Hàm drawKochSnowflake(level):**
 - Dùng để khởi tạo hình tam giác đều ban đầu.
 - Xác định ba đỉnh A, B, C sao cho hình được đặt ở giữa canvas.
 - Bắt đầu đường vẽ tại đỉnh A .
 - Gọi hàm drawKoch cho ba cạnh AB, BC và CA để tạo thành bông tuyết Koch hoàn chỉnh.
- **Hàm drawKoch(p1, p2, level):**
 - Là hàm đệ quy chính dùng để vẽ đường cong Koch trên một đoạn thẳng từ $p1$ đến $p2$.
 - Nếu $level == 0$, chương trình chỉ vẽ trực tiếp đoạn thẳng đến điểm $p2$.
 - Nếu $level > 0$:
 - * Chia đoạn thẳng ban đầu thành ba phần bằng nhau.
 - * Xác định hai điểm chia p_a và p_b .
 - * Tính góc của đoạn thẳng gốc bằng hàm atan2 .
 - * Xác định đỉnh nhô ra peak bằng phép quay một góc $\frac{\pi}{3}$.
 - * Gọi đệ quy cho 4 đoạn con mới tạo thành.
- **Hàm render():**
 - Xóa canvas trước khi vẽ lại.
 - Đọc giá trị cấp độ từ ô nhập liệu.
 - Giới hạn cấp độ trong khoảng từ 0 đến 6 để tránh số lượng đoạn tăng quá lớn, làm giảm hiệu năng hiển thị.
 - Gọi hàm drawKochSnowflake để thực hiện quá trình vẽ.



Hình 1: Sự phát triển của Koch Snowflake qua các cấp độ

1.6 Kết quả

Từ hình 1, có thể quan sát rằng khi cấp độ (level) tăng lên, cấu trúc của Koch Snowflake trở nên ngày càng phức tạp với số lượng đoạn thẳng tăng nhanh theo cấp số mũ. Mỗi đoạn thẳng ban đầu được thay thế bởi bốn đoạn nhỏ hơn, tạo ra các chi tiết hình học tinh vi hơn ở mỗi bước lặp. Đồng thời, mặc dù độ dài mỗi đoạn giảm dần, tổng chu vi của hình lại tăng lên không giới hạn, thể hiện một đặc trưng điển hình của fractal. Tuy nhiên, toàn bộ hình vẫn bị giới hạn trong một vùng không gian hữu hạn, cho thấy sự khác biệt rõ rệt giữa khái niệm độ dài và diện tích trong hình học fractal. Ngoài ra, tính tự đồng dạng của Koch Snowflake cũng được thể hiện rõ ràng, khi mỗi phần nhỏ của đường biên có hình dạng tương tự toàn bộ cấu trúc ở các cấp độ khác nhau.

2 Tìm hiểu về cách vẽ đảo Minkowski

2.1 Giới thiệu

Minkowski Island là một fractal hình học được xây dựng từ một hình vuông ban đầu, trong đó mỗi cạnh của hình được thay thế bằng một đường gấp khúc theo một quy tắc xác định. Khi quá trình này được lặp lại nhiều lần trên toàn bộ các cạnh, biên của hình trở nên ngày càng phức tạp, tạo thành một cấu trúc có tính tự đồng dạng và mang đặc trưng điển hình của fractal.

2.2 Ý tưởng tổng quát

- Bắt đầu với một hình vuông ban đầu.
- Xét từng cạnh của hình vuông.
- Chia mỗi cạnh thành các đoạn nhỏ theo một quy tắc hình học cố định.
- Thay thế cạnh thẳng ban đầu bằng một đường gấp khúc gồm nhiều đoạn con.
- Lặp lại quy trình này trên tất cả các đoạn mới sinh ra theo số cấp độ mong muốn.

2.3 Đặc điểm toán học

- Ở mỗi lần lặp, một đoạn thẳng ban đầu được thay thế bởi 8 đoạn nhỏ hơn.
- Nếu độ dài mỗi đoạn con bằng $\frac{1}{4}$ đoạn ban đầu, thì tổng độ dài đường biên sau mỗi lần lặp tăng theo quy luật:

$$P_n = P_0 \cdot \left(\frac{8}{4}\right)^n = P_0 \cdot 2^n$$

trong đó P_0 là chu vi ban đầu và n là số cấp độ lặp.

- Khi $n \rightarrow \infty$, chu vi của hình tăng không giới hạn, trong khi hình vẫn bị giới hạn trong một miền hữu hạn trên mặt phẳng.
- Số chiều fractal của đường cong Minkowski được xác định bởi:

$$D = \frac{\log 8}{\log 4} = \frac{3}{2}$$

- Fractal này thể hiện rõ tính tự đồng dạng, vì mỗi phần nhỏ của biên có cấu trúc tương tự toàn bộ đường biên ở cấp độ lớn hơn.

2.4 Source code

```
function drawMinkowskiIsland(ctx, level) {
  const size = 300;
  const startX = 150;
  const startY = 150;

  const square = [
    { x: startX, y: startY + size },
    { x: startX + size, y: startY + size },
    { x: startX + size, y: startY },
    { x: startX, y: startY }
  ];

  ctx.beginPath();
  ctx.moveTo(square[0].x, square[0].y);
```



```

for (let i = 0; i < 4; i++) {
  const from = square[i];
  const to = square[(i + 1) % 4];
  const path = generateMinkowskiEdge(from, to, level);

  for (let j = 1; j < path.length; j++) {
    ctx.lineTo(path[j].x, path[j].y);
  }
}

ctx.closePath();
ctx.stroke();
}

function generateMinkowskiEdge(p1, p2, level) {
  if (level === 0) return [p1, p2];

  const dx = p2.x - p1.x;
  const dy = p2.y - p1.y;
  const length = Math.sqrt(dx * dx + dy * dy);

  const dirX = dx / length;
  const dirY = dy / length;
  const segment = length / 4;

  const points = [];

  const A = {
    x: p1.x + dirX * segment,
    y: p1.y + dirY * segment
  };
  points.push(...generateMinkowskiEdge(p1, A, level - 1));

  const B1 = {
    x: A.x + dirX * segment,
    y: A.y + dirY * segment
  };

  const normalDir = {
    x: -dirY,
    y: dirX
  };

  const B2 = {
    x: A.x + normalDir.x * segment,
    y: A.y + normalDir.y * segment
  };
}

```

```

const B3 = {
  x: B2.x + dirX * segment,
  y: B2.y + dirY * segment
};

points.push(...generateMinkowskiEdge(A, B2, level - 1));
points.push(...generateMinkowskiEdge(B2, B3, level - 1));
points.push(...generateMinkowskiEdge(B3, B1, level - 1));

const flipNormal = {
  x: -normalDir.x,
  y: -normalDir.y
};

const C1 = {
  x: B1.x + flipNormal.x * segment,
  y: B1.y + flipNormal.y * segment
};

const C2 = {
  x: C1.x + dirX * segment,
  y: C1.y + dirY * segment
};

const C3 = {
  x: B1.x + dirX * segment,
  y: B1.y + dirY * segment
};

points.push(...generateMinkowskiEdge(B1, C1, level - 1));
points.push(...generateMinkowskiEdge(C1, C2, level - 1));
points.push(...generateMinkowskiEdge(C2, C3, level - 1));

points.push(...generateMinkowskiEdge(C3, p2, level - 1));

return deduplicatePoints(points);
}

function deduplicatePoints(points) {
  const result = [];
  for (let i = 0; i < points.length; i++) {
    if (
      i === 0 ||
      points[i].x !== points[i - 1].x ||
      points[i].y !== points[i - 1].y
    ) {
      result.push(points[i]);
    }
  }
}

```

```

    }
    return result;
}

function render() {
    clearCanvas();

    const level = parseInt(document.getElementById("level").value, 10);
    ctx.strokeStyle = "black";
    ctx.lineWidth = 1.2;

    drawMinkowskiIsland(ctx, Math.max(0, Math.min(level, 4)));
}

```

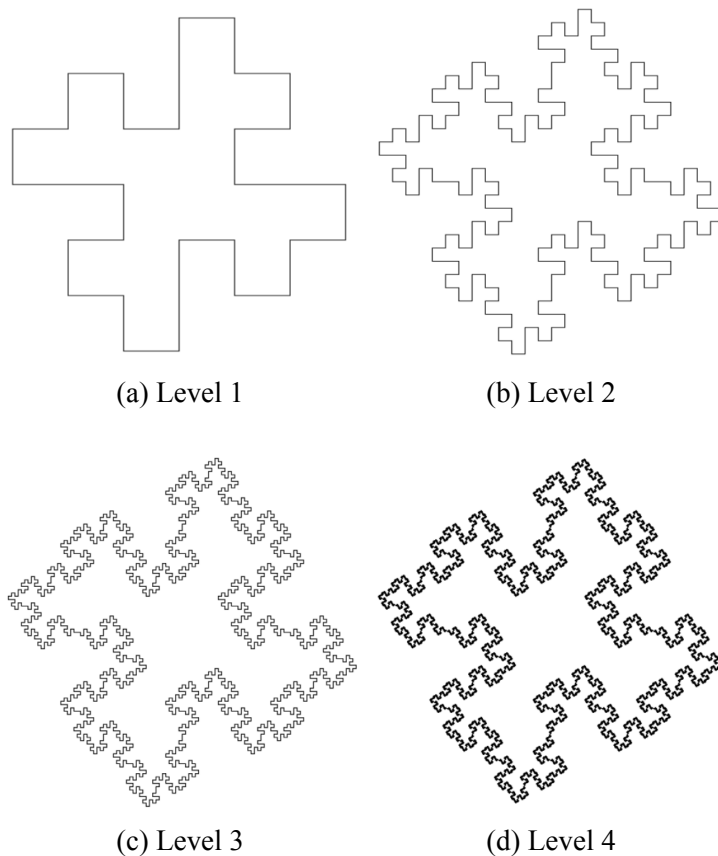
2.5 Phân tích đoạn mã

Đoạn chương trình gồm bốn thành phần chính: hàm vẽ toàn bộ đảo Minkowski, hàm sinh đường Minkowski cho từng cạnh, hàm loại bỏ điểm trùng và hàm điều khiển hiển thị.

- **Hàm `drawMinkowskiIsland(ctx, level)`:**
 - Dùng để khởi tạo hình vuông ban đầu.
 - Xác định bốn đỉnh của hình vuông dưới dạng mảng điểm.
 - Bắt đầu đường vẽ từ đỉnh đầu tiên.
 - Lần lượt xử lý 4 cạnh của hình vuông bằng cách gọi hàm `generateMinkowskiEdge`.
 - Nối các điểm thu được để tạo nên đường biên của đảo Minkowski.
- **Hàm `generateMinkowskiEdge(p1, p2, level)`:**
 - Là hàm đệ quy chính dùng để thay thế một đoạn thẳng bởi một đường gấp khúc Minkowski.
 - Nếu `level === 0`, hàm trả về hai đầu mút của đoạn thẳng ban đầu.
 - Nếu `level > 0`:
 - * Tính vector chỉ phương của đoạn thẳng từ `p1` đến `p2`.
 - * Chia đoạn thẳng thành các phần nhỏ có độ dài bằng $\frac{1}{4}$ đoạn ban đầu.
 - * Xác định các điểm trung gian như `A`, `B1`, `B2`, `B3`, `C1`, `C2`, `C3`.
 - * Sử dụng vector pháp tuyến để tạo các đoạn nhô lên và lõm xuống, từ đó hình thành đường gấp khúc đặc trưng của Minkowski.
 - * Gọi đệ quy trên từng đoạn con để tiếp tục sinh fractal ở cấp nhỏ hơn.
- **Hàm `deduplicatePoints(points)`:**
 - Dùng để loại bỏ các điểm liên tiếp bị trùng nhau trong danh sách điểm.
 - Việc này giúp tránh vẽ lặp lại cùng một điểm, làm đường biên gọn hơn và hạn chế lỗi hiển thị.
- **Hàm `render()`:**

- Xóa canvas trước khi vẽ lại.
- Lấy giá trị cấp độ từ ô nhập liệu.
- Thiết lập màu nét vẽ và độ dày đường biên.
- Giới hạn cấp độ trong khoảng từ 0 đến 4 để tránh số lượng đoạn tăng quá lớn, ảnh hưởng đến hiệu năng hiển thị.
- Gọi hàm drawMinkowskiIsland để thực hiện quá trình vẽ.

2.6 Kết quả



Hình 2: Sự phát triển của Minkowski Island qua các cấp độ

Từ Hình 2, có thể nhận thấy rằng khi cấp độ tăng lên, đường biên của Minkowski Island trở nên ngày càng phức tạp với số lượng đoạn thẳng tăng nhanh. Ở mỗi bước lặp, một đoạn thẳng ban đầu được thay thế bởi nhiều đoạn nhỏ hơn, tạo nên các chi tiết hình học ngày càng tinh vi. Mặc dù độ dài của mỗi đoạn giảm dần, tổng chu vi của hình lại tăng lên theo cấp số mũ, dẫn đến việc đường biên có xu hướng tiến tới vô hạn khi số cấp độ tăng. Tuy nhiên, toàn bộ hình vẫn được giới hạn trong một miền hữu hạn trên mặt phẳng. Ngoài ra, tính tự đồng dạng của fractal cũng được thể hiện rõ ràng qua việc các phần nhỏ của hình có cấu trúc tương tự với toàn bộ hình ở các cấp độ khác nhau. Điều này cho thấy Minkowski Island là một ví dụ điển hình cho sự khác biệt giữa hình học fractal và hình học Euclid truyền thống.

3 Tìm hiểu về cách vẽ tam giác Sierpinski (Triangles) và Hình vuông Sierpinski (Carpet)

3.1 Tam giác Sierpinski (Sierpinski Triangle)

3.1.1 Giới thiệu

Tam giác Sierpinski là một fractal hình học tiêu biểu, được tạo ra thông qua quá trình lặp lại việc phân chia một tam giác đều thành các phần nhỏ hơn. Đặc trưng nổi bật của fractal này là tính tự đồng dạng (self-similarity), tức là cấu trúc của toàn bộ hình vẫn được bảo toàn khi quan sát ở các tỉ lệ thu nhỏ khác nhau.

3.1.2 Ý tưởng tổng quát

- Khởi đầu từ một tam giác đều ban đầu.
- Xác định trung điểm của mỗi cạnh để chia tam giác thành 4 tam giác con.
- Loại bỏ tam giác nằm ở vị trí trung tâm.
- Lặp lại quy trình trên đối với 3 tam giác còn lại.
- Tiếp tục thực hiện cho đến khi đạt cấp độ mong muốn.

3.1.3 Đặc điểm toán học

- Sau mỗi lần lặp, số lượng tam giác con tăng theo quy luật:

$$N = 3^n$$

với n là số cấp độ.

- Diện tích của hình giảm dần theo số lần lặp và tiến dần về 0 khi $n \rightarrow \infty$.
- Độ dài đường biên tăng lên không giới hạn qua các lần lặp, thể hiện một đặc trưng quan trọng của fractal.
- Số chiều fractal của tam giác Sierpinski được xác định bởi:

$$D = \frac{\log 3}{\log 2}$$

3.1.4 Source code

```
function midpoint(p1, p2) {
  return {
    x: (p1.x + p2.x) / 2,
    y: (p1.y + p2.y) / 2
  };
}

function drawSierpinskiTriangle(ctx, p1, p2, p3, level) {
```

```

if (level === 1) {
  ctx.beginPath();
  ctx.moveTo(p1.x, p1.y);
  ctx.lineTo(p2.x, p2.y);
  ctx.lineTo(p3.x, p3.y);
  ctx.closePath();
  ctx.fill();
  return;
}

const m1 = midpoint(p1, p2);
const m2 = midpoint(p2, p3);
const m3 = midpoint(p3, p1);

drawSierpinskiTriangle(ctx, p1, m1, m3, level - 1);
drawSierpinskiTriangle(ctx, m1, p2, m2, level - 1);
drawSierpinskiTriangle(ctx, m3, m2, p3, level - 1);
}

```

3.1.5 Phân tích đoạn mã

- **Hàm midpoint(p1, p2):**
 - Dùng để tính trung điểm của đoạn thẳng nối hai điểm p1 và p2.
 - Kết quả trả về là một điểm mới có tọa độ trung bình theo cả hai trục x và y .
- **Hàm drawSierpinskiTriangle(ctx, p1, p2, p3, level):**
 - Dùng để vẽ tam giác Sierpinski bằng phương pháp đệ quy.
 - Nếu level === 1, chương trình vẽ một tam giác đặc từ ba điểm p1, p2, p3.
 - Nếu level > 1:
 - * Tính trung điểm của ba cạnh.
 - * Chia tam giác ban đầu thành 3 tam giác con cần giữ lại.
 - * Gọi đệ quy trên từng tam giác con với cấp độ giảm đi 1.

3.2 Hình vuông Sierpinski (Sierpinski Carpet)

3.2.1 Giới thiệu

Hình vuông Sierpinski, hay Sierpinski Carpet, là một fractal được xây dựng bằng cách chia một hình vuông thành 9 phần bằng nhau, loại bỏ phần trung tâm và tiếp tục lặp lại quy trình đó trên 8 phần còn lại.

3.2.2 Ý tưởng tổng quát

- Bắt đầu với một hình vuông lớn.
- Chia hình vuông thành lưới 3×3 gồm 9 ô vuông nhỏ bằng nhau.

- Loại bỏ ô vuông ở giữa.
- Lặp lại quy trình trên đối với 8 ô còn lại.
- Tiếp tục thực hiện cho đến khi đạt cấp độ mong muốn.

3.2.3 Đặc điểm toán học

- Sau mỗi lần lặp, số ô vuông còn lại tăng theo quy luật:

$$N = 8^n$$

- Diện tích của hình sau n cấp độ có thể biểu diễn bởi:

$$A_n = \left(\frac{8}{9}\right)^n$$

- Khi $n \rightarrow \infty$, diện tích tiến dần về 0, trong khi đường biên trở nên ngày càng phức tạp.
- Số chiều fractal của Sierpinski Carpet được xác định bởi:

$$D = \frac{\log 8}{\log 3}$$

3.2.4 Source code

```
function drawSierpinski(ctx, x, y, size, level) {
  if (level === 1) {
    ctx.fillRect(x, y, size, size);
    return;
  }

  const newSize = size / 3;
  for (let dx = 0; dx < 3; dx++) {
    for (let dy = 0; dy < 3; dy++) {
      if (dx === 1 && dy === 1) continue;
      drawSierpinski(
        ctx,
        x + dx * newSize,
        y + dy * newSize,
        newSize,
        level - 1
      );
    }
  }
}
```

3.2.5 Phân tích đoạn mã

- Hàm `drawSierpinski(ctx, x, y, size, level)`:
 - Dùng để vẽ fractal Sierpinski Carpet bằng phương pháp đệ quy.
 - Nếu `level === 1`, chương trình vẽ một hình vuông đặc tại vị trí (x, y) với kích thước `size`.
 - Nếu `level > 1`:
 - * Chia hình vuông hiện tại thành 9 ô vuông nhỏ bằng nhau.
 - * Bỏ qua ô trung tâm với điều kiện `dx === 1 && dy === 1`.
 - * Gọi đệ quy cho 8 ô còn lại với kích thước mới bằng $\frac{\text{size}}{3}$.

3.3 So sánh Tam giác Sierpinski và Hình vuông Sierpinski

Thuộc tính	Sierpinski Triangle	Sierpinski Carpet
Hình ban đầu	Tam giác đều	Hình vuông
Số mảnh mỗi lần lặp	3	8
Quy tắc loại bỏ	Tam giác trung tâm	Ô vuông trung tâm
Fractal dimension	≈ 1.585	≈ 1.893

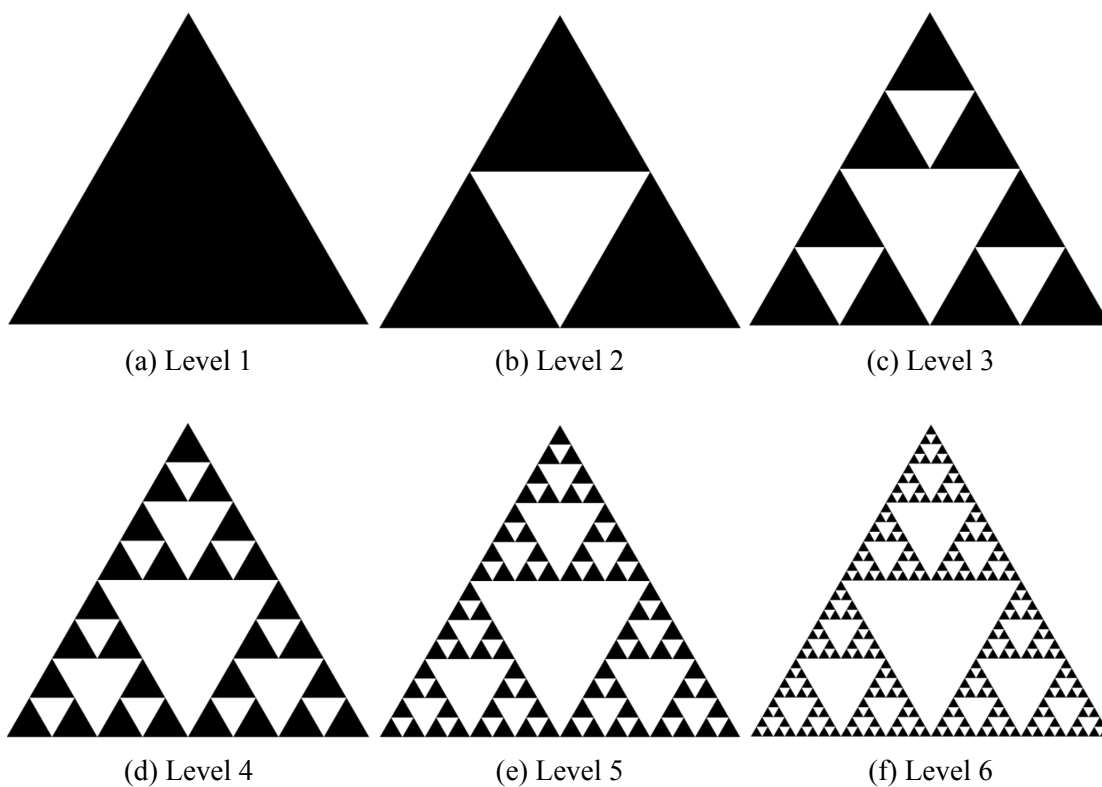
3.4 Kết quả

3.4.1 Sierpinski Triangle

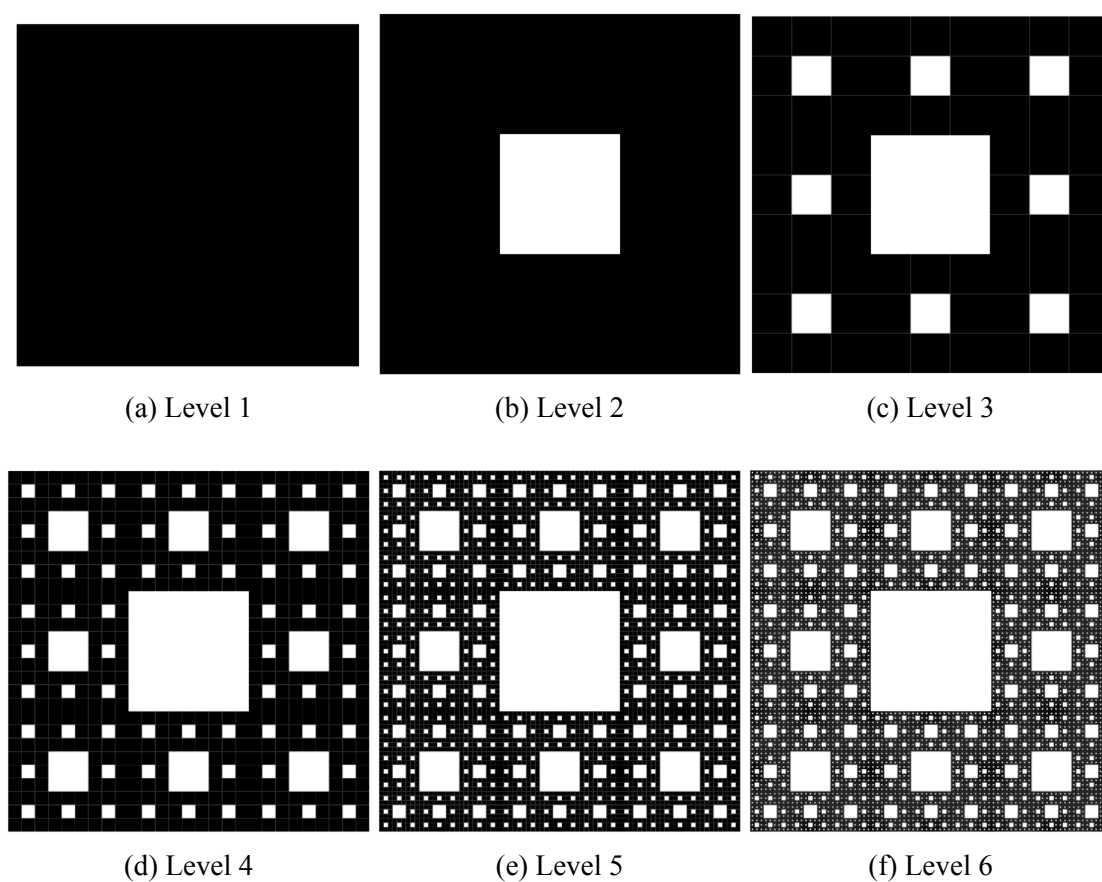
Từ Hình 3, có thể thấy rằng khi cấp độ tăng, tam giác Sierpinski dần hình thành cấu trúc rỗng với các tam giác nhỏ bị loại bỏ lặp đi lặp lại. Số lượng tam giác tăng nhanh theo cấp số mũ, trong khi diện tích tổng thể giảm dần và tiến tới 0. Đồng thời, tính tự đồng dạng được thể hiện rõ khi mỗi phần nhỏ của hình có cấu trúc giống với toàn bộ hình ban đầu.

3.4.2 Sierpinski Carpet

Từ Hình 4, có thể quan sát rằng qua mỗi cấp độ, số lượng ô vuông bị loại bỏ tăng lên đáng kể, tạo nên một cấu trúc ngày càng phức tạp với nhiều khoảng rỗng. Mặc dù diện tích của hình giảm dần, tổng thể hình vẫn giữ nguyên kích thước ban đầu. Tính tự đồng dạng được thể hiện rõ ràng khi mỗi phần nhỏ của hình lặp lại cấu trúc của toàn bộ fractal.



Hình 3: Sự phát triển của Sierpinski Triangle qua các cấp độ



Hình 4: Sự phát triển của Sierpinski Carpet qua các cấp độ

4 Tìm hiểu và trình bày tập Mandelbrot và Julia Set

4.1 Tập Mandelbrot

4.1.1 Giới thiệu

Tập Mandelbrot là một trong những fractal nổi tiếng nhất trong mặt phẳng phức. Tập hợp này được xây dựng từ việc khảo sát sự hội tụ hay phân kỳ của dãy số phức sinh bởi một công thức lặp đơn giản. Mặc dù quy tắc tạo thành khá ngắn gọn, hình dạng của tập Mandelbrot lại rất phức tạp, thể hiện rõ tính chất tự đồng dạng và cấu trúc biên tinh vi của fractal.

4.1.2 Ý tưởng tổng quát

- Xét mỗi điểm $c = x_0 + y_0i$ trên mặt phẳng phức.
- Khởi tạo $z_0 = 0$.
- Lặp theo công thức:

$$z_{n+1} = z_n^2 + c$$

- Nếu dãy $\{z_n\}$ không vượt ra ngoài một ngưỡng nhất định sau số lần lặp cho trước, điểm c được xem là thuộc tập Mandelbrot.
- Ngược lại, nếu dãy phân kỳ, điểm đó không thuộc tập và sẽ được tô màu theo số lần lặp thoát ra ngoài.

4.1.3 Đặc điểm toán học

- Tập Mandelbrot được định nghĩa là tập hợp các số phức c sao cho dãy:

$$z_{n+1} = z_n^2 + c, \quad z_0 = 0$$

là bị chặn.

- Trong thực hành tính toán, điều kiện kiểm tra thường dùng là:

$$|z_n|^2 = x^2 + y^2 \leq 4$$

Nếu giá trị này vượt quá 4, dãy được xem là phân kỳ.

- Biên của tập Mandelbrot có cấu trúc fractal rất phức tạp và chứa nhiều vùng tự đồng dạng.
- Việc tô màu thường dựa trên số lần lặp cần thiết để điểm thoát khỏi ngưỡng phân kỳ, từ đó tạo ra các dải màu thể hiện mức độ hội tụ hoặc phân kỳ của từng điểm.

4.1.4 Source code

```
function drawMandelbrot() {  
  const image = ctx.createImageData(width, height);  
  const data = image.data;  
  
  for (let px = 0; px < width; px++) {
```

```

    for (let py = 0; py < height; py++) {

        let x0 = xmin + (px / width) * (xmax - xmin);
        let y0 = ymin + (py / height) * (ymax - ymin);

        let x = 0;
        let y = 0;
        let iter = 0;

        while (x*x + y*y <= 4 && iter < maxIter) {
            let xtemp = x*x - y*y + x0;
            y = 2*x*y + y0;
            x = xtemp;
            iter++;
        }

        let index = (py * width + px) * 4;

        if (iter === maxIter) {
            data[index] = 0;
            data[index+1] = 0;
            data[index+2] = 0;
            data[index+3] = 255;
        } else {
            let color = iter * 5;

            data[index] = color;
            data[index+1] = color * 0.5;
            data[index+2] = 255 - color;
            data[index+3] = 255;
        }
    }

    ctx.putImageData(image, 0, 0);
}

canvas.addEventListener("click", function(e) {
    let rect = canvas.getBoundingClientRect();

    let x = e.clientX - rect.left;
    let y = e.clientY - rect.top;

    let cx = xmin + (x / width) * (xmax - xmin);
    let cy = ymin + (y / height) * (ymax - ymin);

    let zoom = 0.5;

```

```

let dx = (xmax - xmin) * zoom;
let dy = (ymax - ymin) * zoom;

xmin = cx - dx/2;
xmax = cx + dx/2;
ymin = cy - dy/2;
ymax = cy + dy/2;

drawMandelbrot();
});

drawMandelbrot();

```

4.1.5 Phân tích đoạn mã

- **Hàm drawMandelbrot():**

- Tạo một vùng ảnh mới bằng createImageData để thao tác trực tiếp trên từng điểm ảnh.
- Duyệt qua toàn bộ các điểm ảnh trên canvas bằng hai vòng lặp theo trục x và y .
- Với mỗi điểm ảnh (px, py) , ánh xạ tọa độ điểm ảnh sang tọa độ trên mặt phẳng phức:

$$x_0 = x_{\min} + \frac{px}{width}(x_{\max} - x_{\min}), \quad y_0 = y_{\min} + \frac{py}{height}(y_{\max} - y_{\min})$$

- Khởi tạo $z = 0$ và lặp theo công thức Mandelbrot cho đến khi:
 - * số lần lặp đạt maxIter, hoặc
 - * giá trị $x^2 + y^2 > 4$.
- Nếu điểm không thoát ra ngoài sau số lần lặp tối đa, điểm đó được tô màu đen.
- Nếu điểm phân kỳ, màu sắc được xác định dựa trên số lần lặp iter, tạo hiệu ứng chuyển màu xung quanh biên fractal.

- **Sự kiện click:**

- Khi người dùng nhấp chuột lên canvas, chương trình xác định vị trí chuột trong canvas.
- Tọa độ điểm nhấp được ánh xạ về mặt phẳng phức để tìm tâm phóng to mới.
- Biến zoom = 0.5 làm giảm kích thước của sổ quan sát xuống còn một nửa.
- Các giá trị xmin, xmax, ymin, ymax được cập nhật lại quanh tâm mới.
- Sau đó, hàm drawMandelbrot() được gọi lại để hiển thị vùng đã được phóng to.

4.2 Tập Julia

4.2.1 Giới thiệu

Tập Julia là một họ fractal được xác định từ cùng công thức lặp như tập Mandelbrot, nhưng với một hằng số phức c được cố định trước. Tùy theo giá trị của c , hình dạng của tập Julia có thể thay đổi rất đa dạng, từ những cấu trúc liên thông mềm mại đến những hình phân rã phức tạp. Đây là một trong những fractal quan trọng trong nghiên cứu hệ động lực phức.

4.2.2 Ý tưởng tổng quát

- Chọn trước một hằng số phức $c = c_x + c_y i$.
- Với mỗi điểm ban đầu $z_0 = x + yi$ trên mặt phẳng phức, thực hiện phép lặp:

$$z_{n+1} = z_n^2 + c$$

- Nếu dãy $\{z_n\}$ bị chặn, điểm z_0 được xem là thuộc tập Julia tương ứng với c .
- Nếu dãy phân kỳ, điểm đó không thuộc tập Julia và sẽ được tô màu theo số lần lặp.
- Trong chương trình, giá trị c thay đổi theo vị trí chuột, nhờ đó có thể quan sát nhiều dạng Julia Set khác nhau.

4.2.3 Đặc điểm toán học

- Tập Julia được xây dựng từ phép lặp:

$$z_{n+1} = z_n^2 + c$$

với c là hằng số phức cố định.

- Khác với tập Mandelbrot, ở đây giá trị ban đầu z_0 thay đổi theo từng điểm trên mặt phẳng, còn c được giữ cố định trong mỗi lần vẽ.
- Điều kiện kiểm tra phân kỳ vẫn dựa trên ngưỡng:

$$|z_n|^2 = x^2 + y^2 \leq 4$$

- Hình dạng của tập Julia phụ thuộc mạnh vào giá trị của c . Những thay đổi nhỏ của c có thể tạo ra những biến đổi lớn trong cấu trúc hình học của fractal.

4.2.4 Source code

```
function drawJulia() {  
  const img = ctx.createImageData(width, height);  
  const data = img.data;  
  
  for (let px = 0; px < width; px++) {  
    for (let py = 0; py < height; py++) {  
  
      let x = (px / width) * 3 - 1.5;  
      let y = (py / height) * 3 - 1.5;  
  
      let iter = 0;  
  
      while (x*x + y*y <= 4 && iter < maxIter) {  
        let xtemp = x*x - y*y + cx;  
        y = 2*x*y + cy;  
        x = xtemp;  
        iter++;  
      }  
    }  
  }  
}
```

```

    }

    let i = (py * width + px) * 4;

    if (iter === maxIter) {
        data[i] = 0;
        data[i+1] = 0;
        data[i+2] = 0;
        data[i+3] = 255;
    } else {
        let t = iter / maxIter;

        data[i] = 9*(1-t)*t*t*t*255;
        data[i+1] = 15*(1-t)*(1-t)*t*t*255;
        data[i+2] = 8.5*(1-t)*(1-t)*(1-t)*t*255;
        data[i+3] = 255;
    }
}

ctx.putImageData(img, 0, 0);

document.getElementById("cval").innerText =
    cx.toFixed(3) + " + " + cy.toFixed(3) + "i";
}

canvas.addEventListener("mousemove", (e) => {
    let rect = canvas.getBoundingClientRect();

    let x = e.clientX - rect.left;
    let y = e.clientY - rect.top;

    cx = (x / width) * 2 - 1;
    cy = (y / height) * 2 - 1;

    drawJulia();
});

drawJulia();

```

4.2.5 Phân tích đoạn mã

- Hàm `drawJulia()`:

- Tạo một vùng ảnh mới để thao tác trực tiếp trên từng pixel.
- Duyệt qua toàn bộ các điểm ảnh trên canvas.
- Mỗi điểm ảnh được ánh xạ sang mặt phẳng phức trong vùng:

$$x, y \in [-1.5, 1.5]$$

- Với mỗi điểm ban đầu $z_0 = x + yi$, chương trình thực hiện phép lặp:

$$z_{n+1} = z_n^2 + c$$

trong đó $c = c_x + c_y i$.

- Nếu sau `maxIter` lần lặp mà điểm chưa phân kỳ, điểm đó được tô màu đen.
- Nếu điểm phân kỳ sớm hơn, màu sắc được xác định thông qua tham số chuẩn hóa

$$t = \frac{iter}{maxIter}$$

giúp tạo hiệu ứng màu chuyển mượt hơn so với cách tô màu tuyến tính đơn giản.

- **Sự kiện mousemove:**

- Khi người dùng di chuyển chuột trên canvas, vị trí chuột được ánh xạ thành giá trị mới của hằng số phức c .
- Cụ thể:

$$c_x = \frac{x}{width} \cdot 2 - 1, \quad c_y = \frac{y}{height} \cdot 2 - 1$$

- Sau khi cập nhật c , hàm `drawJulia()` được gọi lại để vẽ ngay hình Julia tương ứng.
- Giá trị hiện tại của c cũng được hiển thị lên giao diện để người dùng dễ theo dõi.

4.3 Mối liên hệ giữa tập Mandelbrot và tập Julia

Tập Mandelbrot và tập Julia đều được xây dựng từ cùng một công thức lặp

$$z_{n+1} = z_n^2 + c$$

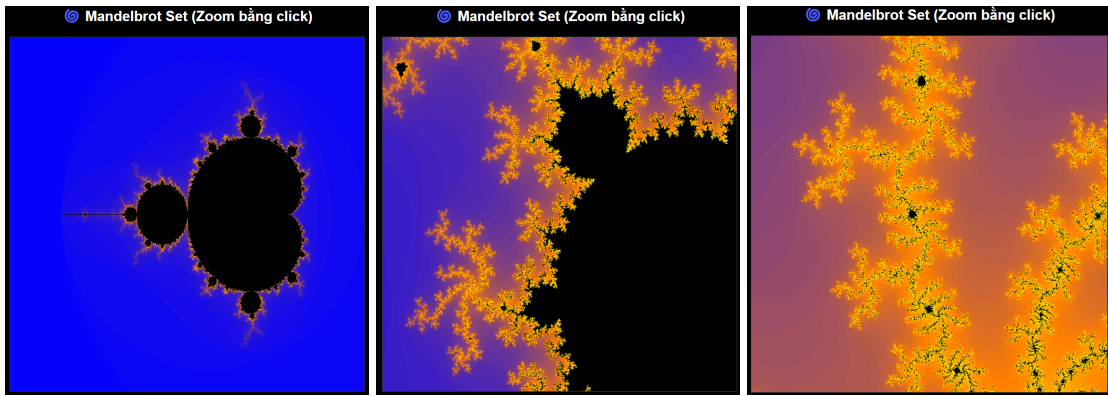
nhưng khác nhau ở cách lựa chọn giá trị ban đầu. Đối với tập Mandelbrot, giá trị c thay đổi trên mặt phẳng phức còn z_0 được cố định bằng 0. Ngược lại, đối với tập Julia, giá trị c được giữ cố định còn z_0 thay đổi theo từng điểm ảnh.

Ngoài ra, tập Mandelbrot còn đóng vai trò như một bản đồ tham số cho họ các tập Julia. Với những giá trị c nằm trong tập Mandelbrot, tập Julia tương ứng thường có cấu trúc liên thông; ngược lại, nếu c nằm ngoài tập Mandelbrot, tập Julia tương ứng thường bị phân rã thành nhiều phần rời rạc. Điều này cho thấy mối liên hệ chặt chẽ giữa hai loại fractal trong hệ động lực phức.

4.4 Kết quả

4.4.1 Mandelbrot

Từ quá trình hiển thị được minh họa trong hình 5, có thể thấy tập Mandelbrot có biên rất phức tạp và xuất hiện nhiều cấu trúc lặp lại ở các mức thu phóng khác nhau. Khi thực hiện thao tác zoom, những chi tiết mới tiếp tục xuất hiện quanh biên của hình, thể hiện rõ đặc trưng tự đồng dạng và độ phức tạp vô hạn của fractal. Việc tô màu theo số lần lặp giúp phân biệt rõ các vùng hội tụ và phân kỳ trên mặt phẳng phức.

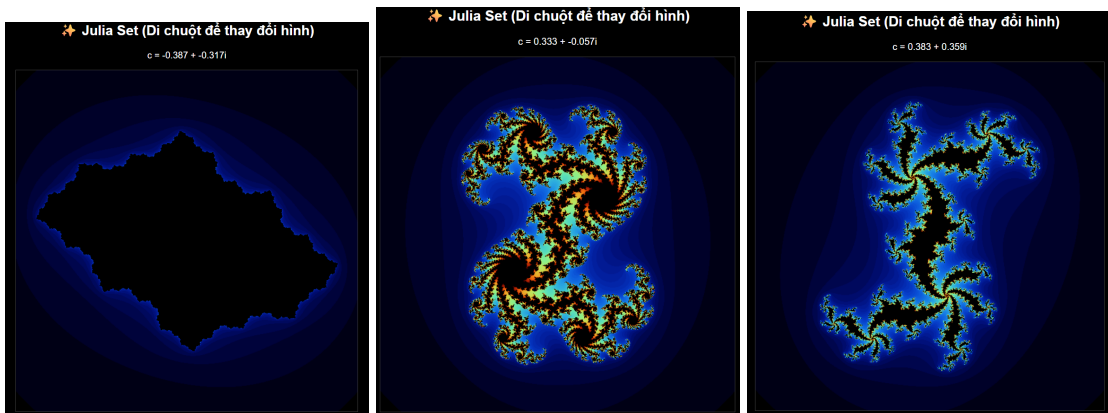


(a) Hình 5.1

(b) Hình 5.2

(c) Hình 5.3

Hình 5: Minh họa Mandelbrot Set



(a) Hình 6.1

(b) Hình 6.2

(c) Hình 6.3

Hình 6: Minh họa Julia Set

4.4.2 Julia

Tập Julia được minh họa trong hình 6 cho thấy sự đa dạng rất lớn về hình dạng khi thay đổi hằng số phức c . Trong chương trình, chỉ cần di chuyển chuột để thay đổi c , hình fractal đã có thể biến đổi đáng kể. Điều này thể hiện tính nhạy cảm cao của hệ động lực phức đối với tham số đầu vào. Việc cho phép tương tác trực tiếp giúp quan sát rõ hơn mối liên hệ giữa giá trị c và cấu trúc hình học của tập Julia.

5 Source code và tài liệu tham khảo

- Source code: [GitHub Repository](#)
- Video demo: [Link dẫn đến drive video demo](#)
- Tài liệu tham khảo
 - <https://adammurray.link/webgl/#basic-examples>
 - <http://www.shodor.org/interactivate/activities/KochSnowflake/>
 - <http://www.shodor.org/interactivate/activities/SierpinskiTriangle/>